

A4 神经网络 从神经元到矩阵运算

把小计算连接起来 并用数字表整理重复计算

这是 A4 的基础部分，另一份 A4《激活函数与 SGD》可在读完后选学。先复习 A2 的向量、点积与梯度下降，以及 A3 的训练循环。遇到激活函数和矩阵可以手算小数字，但不要求先学过微积分。

读完你会知道

不必先背公式。下面用同一个两输入的小例子反复计算：先看一个神经元如何得出数字，再把三个神经元并排写成矩阵，最后把一次预测放回训练循环。读完后，你应能解释每个符号代表什么，并能检查一层网络的矩阵维度。

已经学过	这一节怎样使用它
A2	特征向量、点积、损失、梯度下降。
A3	前向传播、反向传播、batch、step、epoch。
这一节	把上述概念放进神经网络，并重点理解“一个单元”和“一层”的数学形式。

1 先建立一张地图

为什么线性回归之后还要学神经网络？一条直线很适合说明基本训练过程，但复杂的图片或文字关系往往难用一次简单加权来表达。神经网络把许多可调整的小计算连接起来，逐层形成更有用的表示。

机器学习是让算法从数据中学习规律的总称。神经网络是其中一类模型；它由许多简单计算单元连接而成，数据经过很多的“层”进行处理。使用较多层神经网络的方法通常称为深度学习。这里的“神经元”是受生物神经元启发的数学名称，不是真实脑细胞。

一次预测可以看成一条流水线：输入层接收数字；隐藏层把数字变成新的表示；输出层给出最终预测。输入层通常不做复杂计算，它只是把特征交给后面的计算单元。

这一节将沿用 x 表示输入， w 、 W 和 b 表示要调整的参数。先从一个具体计算开始，遇到新的字母时再解释它的含义；不需要先记住一整张符号表。

2 一个神经元到底算了什么

为什么要从一个计算单元开始？整张网络的公式看起来很大，但最基本的一步仍是把输入分别乘上可学习的权重，再加上偏置。先手算这一小步，后面的一层和多层才有依据。

先处理一个样本。假设我们用两个输入 x_1 和 x_2 来预测一个结果。神经元为每个输入配一个权重，然后把乘积相加，再加上偏置。加权求和的结果暂时记作 z 。

$$z = w_1 x_1 + w_2 x_2 + b$$

把它写成向量形式就是点积。 x 是输入列向量， w 是权重列向量； $w^T x$ 表示把对应位置相乘后求和。

$$x = [x_1, x_2]^T, \quad w = [w_1, w_2]^T, \quad z = w^T x + b$$

手算一次

令 $x_1=2$, $x_2=3$, $w_1=1$, $w_2=0.5$, $b=0.1$ 。每一个数字都有明确角色：2 和 3 来自样本；1、0.5 和 0.1 是当前模型参数。

$$z = 1 \times 2 + 0.5 \times 3 + 0.1 = 3.6$$

偏置 b 的作用是平移计算结果。即使所有输入都是 0， z 仍可等于 b ；因此模型不必被迫经过原点。

3 为什么 z 后面还要接激活函数

函数就是按确定规则把输入变成输出的对应关系，例如 $f(x) = 2x + 1$ 。这里 f 是规则的名字， $f(3)$ 就是把 3 代进去，得到 7。“线性”在这一节直观地指直线式关系：输入每增加相同数量，输出变化也相同；严格数学中，含偏置的“乘后再加常数”叫仿射变换。这一节只需理解它仍是一条直线。

如果每一层都只做“乘以权重、加上偏置”，无论叠多少层，整体仍等价于一层线性变换。例如 $h=2x+1$ 、 $y=3h+4$ ，代入后 $y=6x+7$ 。层数增加了，但关系仍是一条直线。

若增加一条“负数变成 0，非负数保留”的规则， z 从 -2 到 -1 时输出一直是 0，从 1 到 2 时输出却增加 1，整个关系就不再是一条直线。这种不服从单一直线关系的变化叫非线性。接在加权求和后面的转换规则叫激活函数，用 g 命名； $a = g(z)$ 表示把 z 按这条规则转换后得到 a 。

$$z = w^T x + b, \quad a = g(z)$$

刚才这条规则的名字是 ReLU，读作“瑞露”。写成 $\max(0, z)$ 时， $\max$ 表示取括号中较大的数。先掌握这个能手算的规则；下面两个输出规则只在需要判断类别时使用。

表示某个候选有多大可能的数叫概率，范围从 0 到 1；0.7 就是 70%。模型给出的概率是它算出的倾向，不是对现实的保证。“猫或非猫”有两种选择，叫二分类；“苹果、梨、香蕉”有三种选择，属于多分类。一次只能选一种的类别叫互斥类别。

有时我们要把一个任意大小的分数变成 0 到 1 之间的数。一条常用规则叫 Sigmoid：分数 0 变成 0.5，分数 2 约变成 0.881，分数 -2 约变成 0.119。它可用于二分类输出；例如用一项表示“是猫”的概率，剩余概率表示“不是猫”。这里先用给定结果，不要求背公式。

若有多个互斥候选，就需要一组相加等于 1 的数。一条常用规则叫 Softmax：先把每个分数转换成正数，再分别除以这些正数的总和。例如分数 [2,1,0] 先变成约 [7.389,2.718,1]，总和约 11.107；分别相除约得 [0.665,0.245,0.090]。下面解释这些正数从哪里来。

计算候选概率时会用到固定常数 $e \approx 2.718$ 的乘方。 e^2 表示 $e \times e \approx 7.389$ ， $e^1 = e$ ， $e^0 = 1$ ；负指数表示倒数，例如 $e^{-1} = 1/e \approx 0.368$ 。小数指数可由计算器得到，这一节可直接使用提供的近似值。无论原分数正负， e 的乘方都为正，所以相除后适合当作候选概率。

函数	规则	适合怎样理解
ReLU	$g(z) = \max(0, z)$	正数原样通过，负数变为 0。隐藏层常用。
Sigmoid	例如分数 $0 \rightarrow 0.5$ ， $2 \rightarrow$ 约 0.881	输出在 0 到 1 之间；常用于二分类概率式输出。
Softmax	各分数先变成 e 的乘方，再除以总和	多分类输出层使用；它处理的是一整组分数，不是单个数。

例如 $z=3.6$ 时， $\text{ReLU}(3.6)=3.6$ ； $z=-2$ 时， $\text{ReLU}(-2)=0$ 。注意：负数被截为 0 只是 ReLU 的规则，不表示这个输入“没有意义”。

4 三个神经元并排 就是一层

同一份输入可以同时交给多个神经元。它们看到相同的 x_1 、 x_2 ，但各自拥有不同的权重和偏置，所以会学到不同的变换。三个神经元并排计算，就称为一个有 3 个单元的层。

神经元	权重和偏置	对 $x=[2,3]^T$ 的 z
第 1 个	$w=[1, 0.5]^T$ ， $b=0.1$	$1 \times 2 + 0.5 \times 3 + 0.1 = 3.6$

神经元	权重和偏置	对 $x=[2,3]^T$ 的 z
第 2 个	$w=[-1, 1]^T, b=0$	$-1 \times 2 + 1 \times 3 + 0 = 1$
第 3 个	$w=[0.2, -0.4]^T, b=0.5$	$0.2 \times 2 - 0.4 \times 3 + 0.5 = -0.3$

若这一层使用 ReLU，它会把三个 z 值逐个变成激活值： $a=[3.6, 1, 0]^T$ 。这个三维向量就是这一层交给下一层的输出。

5 用矩阵把整层一次写完

为什么改用矩阵？三个神经元逐个写三条相似公式，容易抄错，也不方便同时处理更多神经元。矩阵把这些计算按行排好，一次写出整层结果。

矩阵是按行列排列的数字表。 3×2 的“形状”表示 3 行、2 列，不是说它等于数字 6。矩阵乘列向量时，用表里的每一行和输入做一次点积：对应数相乘再相加；每一行算出一个输出。所以这里 3 行就会产生 3 个数。

若右边也是矩阵，就对它的每一列分别做上述计算。例如 $[[1,2],[3,4]]$ 乘 $[[5,6],[7,8]]$ ：第一列结果是 $[1 \times 5 + 2 \times 7, 3 \times 5 + 4 \times 7]^T = [19, 43]^T$ ；第二列是 $[22, 50]^T$ 。合起来得到 $[[19, 22], [43, 50]]$ 。它不是两个矩阵对应位置逐项相乘。

上面的三次计算结构完全相同，因此可以把每个神经元的权重排成矩阵 W ，把三个偏置排成向量 b 。约定“每一行对应一个神经元”：3 个神经元、每个接收 2 个输入，所以 W 的形状是 3×2 。

$$W = [[1, 0.5], [-1, 1], [0.2, -0.4]], \quad b = [0.1, 0, 0.5]^T$$

$$z = Wx + b = [3.6, 1, -0.3]^T, \quad a = \text{ReLU}(z) = [3.6, 1, 0]^T$$

维度检查

对象	形状	为什么
输入 x	2×1	两个输入特征排成一列。
权重 W	3×2	三行对应三个神经元；两列对应两个输入。
Wx	3×1	每行 2 个权重配对 2 个输入，3 行各得到一个数。

对象	形状	为什么
偏置 b	3×1	每个神经元各有一个偏置，可以与 Wx 相加。
激活 a	3×1	激活函数逐项处理 z ，形状不变。

维度不匹配时，先不要急着算。检查“前一个对象的列数是否等于后一个对象的行数”。这是发现矩阵公式写错的最快办法。

6 从一层走到多层

第一层输出 a_1 不再是原始特征，而是网络自己计算出的新表示。第二层把 a_1 当作输入，使用自己的 W_2 和 b_2 。下标只表示层号，不是乘法。

$$z_1 = W_1x + b_1, \quad a_1 = \text{ReLU}(z_1)$$

$$z_2 = W_2a_1 + b_2, \quad \hat{y} = g(\text{输出}(z_2))$$

这里的 $\hat{y}$ 读作“y hat”，表示预测值。回归问题的输出层可以直接给数值；二分类常用一个 Sigmoid 输出；多分类常用 Softmax 把多个类别分数变成一组概率。输出层的激活函数应由任务决定，不能只因为隐藏层用了 ReLU 就照搬。

7 一批样本如何写成矩阵

为什么还要把样本排在一起？训练时反复逐个写同一层公式很繁琐。将多个样本按列放进 X ，就能用同一组权重同时计算它们，并清楚检查输入与输出的形状。

真实训练通常一次处理多个样本。沿用“每一列是一个样本”的约定：如果 batch 内有 B 个样本，每个样本有 d 个特征，那么输入矩阵 X 的形状是 $d \times B$ 。一个含 h 个神经元的层使用 W ($h \times d$) 和 b ($h \times 1$)。

$$Z = WX + b, \quad A = g(Z)$$

这里 b 会复制到 B 列后与 WX 相加；许多编程框架把这种按维度自动扩展的规则叫 broadcasting。它并不是多了一份新的偏置参数，而是同一个 b 用于 batch 中的每个样本。

量	形状	含义
X	$d \times B$	B 个样本，每列一个样本。
W	$h \times d$	h 个神经元，每个读取 d 个输入。
WX 和 Z	$h \times B$	每列是一个样本在这一层的激活前值。
b	$h \times 1$	每个神经元一个偏置，复制到 B 列。
A	$h \times B$	激活后的整批输出。

8 预测怎样变成学习

A3 的训练循环在神经网络中完全一样，只是参数从线性回归的 w 、 b 扩展为每一层的 W 、 b 。前向传播按上面的矩阵公式得到预测；损失函数比较预测和正确答案；反向传播利用链式法则高效地算出每个参数的梯度；优化器沿负梯度更新参数。

$$W \leftarrow W - \eta \partial L / \partial W, \quad b \leftarrow b - \eta \partial L / \partial b$$

η 是学习率， L 是当前 batch 的平均损失。符号 $\partial L / \partial W$ 的意思是：“如果稍微改变 W ，损失 L 会怎样变化？”反向传播不是另一种模型，而是重复利用前向计算过程来求这些梯度的方法。

如果每次用全部训练样本算一次梯度再更新，叫全批量梯度下降。如果每次随机取一个样本更新，叫随机梯度下降，英文缩写 SGD；随机是为了减少总按同一顺序取样造成的影响。如果每次取一小批，就叫小批量随机梯度下降，常写作 mini-batch SGD，其中 mini-batch 就是小批量。

名称	一次更新用多少训练样本	特点
全批量梯度下降	全部训练集	方向较稳定，但每一步成本高。
严格 SGD	1 个样本	一次计算量较小，但相邻更新方向可能差别较大。
mini batch SGD	一小批样本	兼顾效率和稳定性，实践中最常见。

很多代码库把 mini-batch 训练也简称为 SGD。看到这个词时，最好继续确认 batch size；batch size=1 才是严格意义上的单样本 SGD。

9 把整件事串起来

对一个 mini-batch，模型先把 X 经过每层的 W 、 b 和激活函数，得到预测 $\hat{y}$ 。接着用标签 y 计算损失 L 。反向传播得到每个 W 、 b 的梯度；优化器更新一次参数，这就是一个 step。训练集大致完整地看一遍是一个 epoch。验证集用于选择设置和保存较好的模型；测试集留到最后一次评估。

可选挑战 多分类输出

三个互斥类别的分数是 $[2, 1, 0]$ 。请利用这一节给出的正数 $[7.389, 2.718, 1]$ ，算出它们的总和，并说明最大概率对应哪个类别、概率为何相加为 1。若分别给三个类别做 Sigmoid，为什么不能保证相加为 1？

参考答案：总和约 11.107，最大分数 2 对应约 $7.389 \div 11.107 = 0.665$ ，也是最大概率。每项都除以同一总和，所以相加为 1。单独做 Sigmoid 时，各项没有共同除以它们的总和；例如三个分数都是 0，会分别得到 0.5，总和 1.5。

自己试一试

给定 $x=[1, 4]^T$ ， $w=[2, -0.5]^T$ ， $b=0.3$ 。请依次计算：① z ；② $\text{ReLU}(z)$ ；③ 若把这个神经元作为一层， x 、 w 、 b 、 z 各是什么形状？④ 若学习率 $\eta=0.1$ ，且 $\partial L / \partial w = [3, -2]^T$ ，更新后的 w 是什么？

参考答案： $z=2 \times 1 + (-0.5) \times 4 + 0.3 = 0.3$ ， $\text{ReLU}(z)=0.3$ ； x 、 w 都是 2×1 ， b 、 z 都是 1×1 ； $w_{\text{new}} = [2, -0.5]^T - 0.1[3, -2]^T = [1.7, -0.3]^T$ 。

一页记忆

一个神经元先算 $z=w^T x + b$ ，再算 $a=g(z)$ 。一层神经元并排时， $z=Wx+b$ ， $a=g(z)$ ； W 的每一行对应一个神经元。多层网络把上一层的 a 当作下一层的输入。训练时，前向传播得到预测，损失衡量误差，反向传播求梯度，mini-batch SGD 更新 W 和 b 。